

JavaScript Prototypes & Inheritance

Fundamentals

1. What is a prototype, and why use prototype-based inheritance?

A prototype is an internal object from which other objects inherit properties. JavaScript uses prototype-based inheritance to share behavior (methods) and properties across multiple object instances without duplicating them in memory. Instead of copying a class blueprint for every instance, instances maintain a live link to their prototype. This makes inheritance highly dynamic and memory-efficient.

2. Difference between `[[Prototype]]`, `prototype`, and `__proto__`?

`[[Prototype]]` : This is the internal, hidden slot (as defined in the ECMAScript spec) on an object that points to its actual prototype object. It cannot be accessed directly.

`__proto__` : This is an accessor property (getter/setter) historically exposed by browsers on `Object.prototype` that allows you to read or mutate an object's internal `[[Prototype]]` . It is largely deprecated in favor of `Object.getPrototypeOf()` .

`prototype` : This is a standard property found *only* on constructor functions (and classes). It is the object that will be assigned as the `[[Prototype]]` of all instances created when that function is invoked with `new` .

3. Relationship between an object and `Object.prototype`?

By default, almost all objects created in JavaScript (via object literals `{}` or `new Object()`) inherit from `Object.prototype` . It forms the terminus of the standard prototype chain. Its own internal `[[Prototype]]` is `null` . Thus, methods like `toString()` and `hasOwnProperty()` exist on `Object.prototype` and are available to all standard objects.

4. Property lookup when an object doesn't own a property?

When you access a property (e.g., `obj.prop`), the engine first checks if `obj` has it as an *own* property. If not, it delegates the lookup to the object's `[[Prototype]]` . If the prototype object doesn't have it, it checks the prototype's prototype, traversing up the chain until it either finds the property or reaches `null` (returning `undefined`).

5. Prototype delegation vs. copying inherited properties?

In classical copying, inheriting properties means structurally copying methods into every new instance's memory space. In prototype delegation, properties are not copied. The instance remains empty of inherited methods; it merely holds a reference. When a method is called, the execution is delegated "up the chain," while the `this` context dynamically remains bound to the original instance.

6. Instance's prototype vs. constructor's prototype property?

An instance's prototype (accessed via `Object.getPrototypeOf(instance)`) is the object it inherits from. The constructor's `prototype` property (e.g., `User.prototype`) is a template object. When `new User()` is called, the instance's internal `[[Prototype]]` is explicitly set to point to the `User.prototype` object.

7. Object.create() vs. constructing with new?

`Object.create(proto)` creates a brand new, empty object and directly assigns its internal `[[Prototype]]` to the `proto` object passed as an argument. It bypasses constructor logic completely.

`new Constructor()` creates a new object, links its prototype to `Constructor.prototype`, and executes the constructor function to initialize instance variables, binding `this` to the new object.

8. Why do arrow functions lack a prototype property?

Arrow functions were designed to be lightweight, lexically bound closures, not object constructors. To save memory and initialization time, the JavaScript engine does not create a `prototype` object for them, and they lack the internal `[[Construct]]` method. Consequently, using `new` on an arrow function throws a `TypeError`.

Intermediate

1. What happens internally when new Constructor() executes?

Four things happen internally:

- A new, empty ordinary object is created.
- The new object's internal `[[Prototype]]` is set to the `Constructor.prototype` object.
- The `Constructor` function is executed, with `this` bound to the newly created object.
- If the constructor explicitly returns a non-primitive object, that object is returned. Otherwise, the newly created object is returned.

2. How does instanceof determine instance matching?

The `instanceof` operator (e.g., `a instanceof User`) checks the prototype chain. It verifies if the exact object reference held in `User.prototype` currently appears anywhere in the internal `[[Prototype]]` chain of object `a`. It does not verify how the object was originally created.

3. Replacing Constructor.prototype vs. existing instances?

If you reassign a constructor's prototype (`Constructor.prototype = {}`), existing instances are unaffected. Their internal `[[Prototype]]` reference points to the *original* object in memory that existed at the time of their creation. New instances created after the reassignment will point to the new prototype object, fragmenting the inheritance tree.

4. Adding a method vs. replacing the prototype?

Mutating the existing prototype (`User.prototype.greet = ...`) updates the shared object in memory. Because inheritance is a live link, all existing instances instantly gain access to the new method. Replacing the prototype entirely breaks the link for existing instances, as explained above.

5. Shadowing and property lookup

Shadowing occurs when an object defines a property with the exact same name as a property on its prototype chain. During lookup, the engine stops at the first match. The instance's own property "shadows" the inherited property, preventing the prototype's property from being accessed directly via that instance.

6. Assigning to inherited getter, setter, or non-writable property?

If a prototype has a property marked `writable: false` , assigning to that property on the instance will silently fail (or throw in strict mode)—it does *not* create a shadowing property. If the prototype defines a setter for that key, assigning to the instance invokes the prototype's setter rather than creating an own property.

7. getPrototypeOf() vs. setPrototypeOf() vs. Object.create()

`Object.getPrototypeOf(obj)` reads the internal prototype.

`Object.setPrototypeOf(obj, proto)` mutates an existing object's prototype (highly discouraged for performance reasons).

`Object.create(proto)` creates a *new* object with the specified prototype, which is the performant and safe way to establish chains manually.

8. How class extends establishes dual inheritance?

The `class Child extends Parent` syntax sets up two prototype chains simultaneously. First, it links the instance prototypes: `Object.setPrototypeOf(Child.prototype, Parent.prototype)` so instances inherit methods. Second, it links the constructors themselves: `Object.setPrototypeOf(Child, Parent)` , allowing the child class to inherit static methods from the parent class.

Advanced

1. Implementing method sharing without duplicating allocations?

To avoid duplicating function objects per instance, methods should be attached to the prototype object rather than defined inside the constructor.

```
function User(name) { this.name = name; }  
// Method allocated once in memory  
User.prototype.greet = function() { return this.name; };
```

2. Inheriting from multiple objects (Multiple Inheritance)?

JavaScript's internal `[[Prototype]]` is a single pointer; an object cannot inherit directly from multiple prototypes. To compose behavior from multiple sources, developers use **Mixins**. You dynamically copy methods from multiple source objects into a single target prototype using `Object.assign()` .

3. instanceof across browser frames or realms?

Different browser frames (or iframes/realms) have their own separate global environments, meaning they each have their own `Array`, `Object`, and custom constructors. If you pass an array from an iframe to the parent window, `arr instanceof Array` evaluates to `false` because the parent's `Array.prototype` is a physically different object in memory than the iframe's `Array.prototype`.

4. Constructor explicitly returning an object or primitive?

If a constructor explicitly returns a non-primitive object (e.g., `return { custom: true }`), the `new` operator will return that object instead of the newly constructed instance, and the `this` binding is discarded. If the constructor returns a primitive (string, number, boolean, null), the engine ignores the return statement and returns the newly constructed instance object.

5. Determining if a property is owned vs. inherited?

You use `Object.hasOwn(obj, 'prop')` (or the older `obj.hasOwnProperty('prop')`). This method returns `true` only if the property exists physically on the object itself, and `false` if the property is missing or resides on the prototype chain. The `in` operator, conversely, returns true for both owned and inherited properties.

6. Why modifying Object.prototype breaks code?

Modifying `Object.prototype` (e.g., `Object.prototype.myMethod = ...`) leaks the property into the prototype chain of nearly every object in the application. Standard `for...in` loops iterate over enumerable inherited properties. Unless strictly defined with `enumerable: false`, adding to `Object.prototype` will cause arbitrary objects (and third-party library internals) to spontaneously yield the injected keys during iteration, causing crashes.

7. Prototype mutation and method resolution?

Because method resolution happens at the exact moment of invocation (late binding), mutating a prototype dynamically alters the behavior of all inherited instances instantly. If an instance relies on `data.parse()`, and you hot-swap `Data.prototype.parse`, the next time the instance calls it, it will execute the new logic.

8. Composition, Map, or null-prototype dictionaries?

Prefer composition over deep inheritance trees to avoid the "Gorilla Banana" problem (inheriting a banana, but getting the gorilla and the whole jungle). Use `Map` or `Object.create(null)` for pure key-value stores. A null-prototype object prevents prototype pollution and key collisions (like a user submitting a payload with a key named `__proto__` or `toString`).

Deep-Dive Internals

1. Prototype chains vs. Hidden Classes / Object Shapes?

V8 and other modern engines optimize property access using "Hidden Classes" (Shapes). Objects with the same layout share a Shape. The engine caches the offset of properties (Inline Caches). The prototype is heavily tied to an object's Shape. If instances share a prototype and property initialization order, they share a Shape, making method lookups extremely fast. Modifying the prototype or dynamically adding properties to instances causes Shape transitions, breaking Inline Caches and severely degrading performance.

2. The performance impact of `Object.setPrototypeOf()`

Calling `Object.setPrototypeOf()` forces the JS engine to discard the optimized Hidden Shape of the target object. Furthermore, because prototype links are heavily optimized for read-access, mutating the chain forces the engine to flush all Inline Caches across the entire application that ever touched objects related to that prototype chain. The object is permanently marked as a "dictionary mode" slow-object.

3. Unsafe recursive merge and Prototype Pollution

Deep merge functions (like `_.merge`) iterate over keys in a JSON payload and assign them to a target object. If a malicious payload includes the key `"__proto__"`, a naive merge will interpret this as a normal string key. It recursively modifies the `Object.prototype` of the Node.js server or browser environment. An attacker can inject properties (e.g., `isAdmin: true`) that pollute the entire application globally. **Secure alternative:** Filter out keys named `__proto__`, `constructor`, and `prototype` during recursion, or use `Map`.

4. Proxy invariants for prototype operations

When using Proxies to intercept `getPrototypeOf` or `setPrototypeOf`, the ECMAScript spec enforces strict invariants. If the underlying target object is non-extensible (e.g., via `Object.preventExtensions()`), a Proxy's `getPrototypeOf` trap must return exactly the true `[[Prototype]]` of the target. It cannot lie. Attempting to return a spoofed prototype for a non-extensible object throws a `TypeError` to ensure engine security guarantees.

5. 'super' method resolution and `[[HomeObject]]`

`super.method()` is not equivalent to `this.__proto__.method()`. If it were, calling `super` inside an inherited method would result in an infinite loop due to `this` always pointing to the base instance. Instead, ES6 class methods are bound to a hidden `[[HomeObject]]` (the prototype they were defined on) at creation time. `super` statically looks up the prototype of the `[[HomeObject]]`, ensuring method resolution predictably climbs the chain.

6. Instance chain vs. Constructor chain (Dog extends Animal)

Instance Chain: `Dog.prototype` has an internal `[[Prototype]]` pointing to `Animal.prototype`. This dictates how a `new Dog()` instance resolves inherited standard methods.

Constructor Chain: The `Dog` function object itself has an internal `[[Prototype]]` pointing to the `Animal` function object. This is how static methods (e.g., `Animal.create()`) are inherited and accessible via `Dog.create()`.

7. Ordinary vs. Exotic Objects prototype behavior

Ordinary objects follow the standard ECMAScript specification for `[[GetPrototypeOf]]` and `[[SetPrototypeOf]]` algorithms. Exotic objects (like Proxy objects, Module namespaces, or the global `Window` object) override these internal methods. For example, a Proxy handles prototype requests via its traps, and `String` objects have specialized exotic behavior for indexed properties.

Output-Based Questions

33. Output and Explanation

```
const a = new User();
User.prototype.greet = () => "Hello";
console.log(a.greet());
console.log(a instanceof User);
```

Output: `"Hello"` and `true`.

Why: Prototypes are live references. Even though `a` was instantiated before `greet` was added to the prototype, the lookup for `a.greet` happens at call time, traversing the live chain to find the newly added method. `instanceof` checks if the current `User.prototype` object exists in `a`'s prototype chain, which it does.

34. Output and Explanation (Assuming generic instantiation)

```
const a = new User();
```

Context: When this executes, the engine allocates an object, sets its `[[Prototype]]` to `User.prototype`, and evaluates the `User` constructor. If `User` does not exist, it throws a `ReferenceError`. If `User` lacks a `[[Construct]]` slot (e.g., an arrow function), it throws a `TypeError`.